Lista Encadeada

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

Uso de arrays em estruturas lineares

Acesso direto (eficiência)

Alocação em tempo de compilação (desperdício de memória)

Listas ligadas (linked lists)

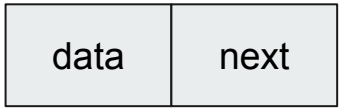
TAD fundamental

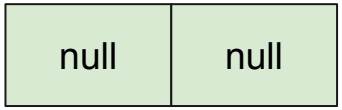
Tamanho arbitrário e dinâmico

Acesso sequencial

Permite elementos duplicados

Listas ligadas (Design)

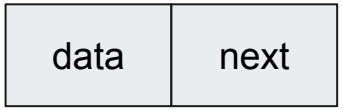




head

head

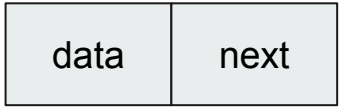
Listas ligadas (Design)

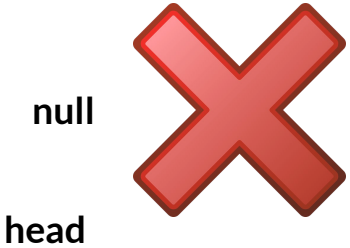


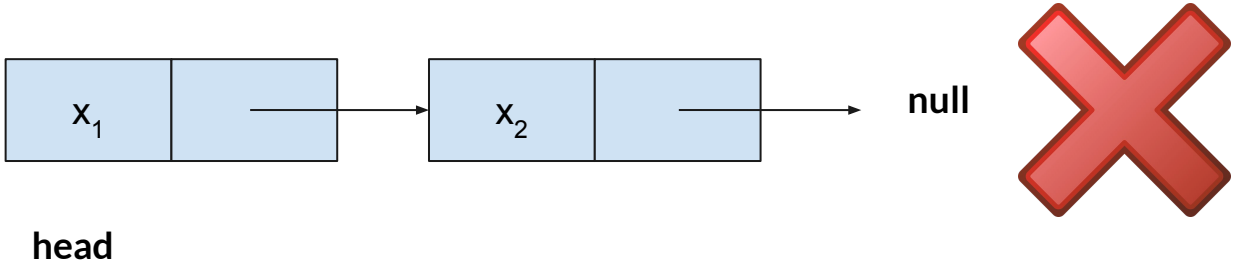
head

head

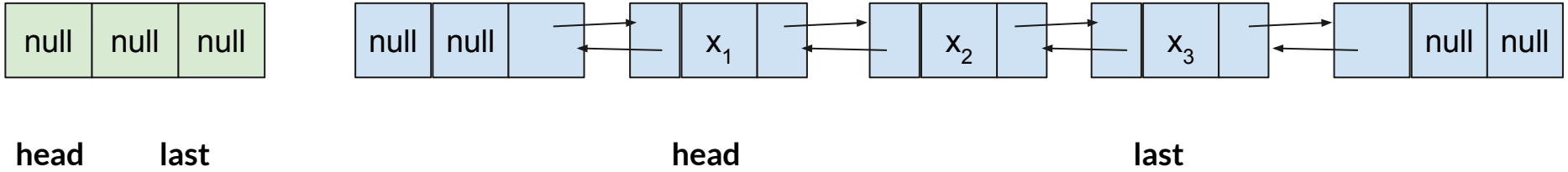
Listas ligadas (Design)



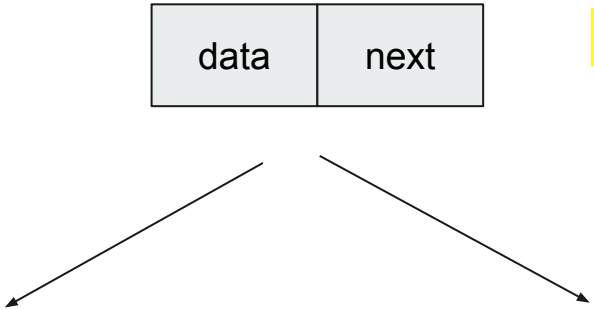




Listas duplamente ligadas (Design)



Listas ligadas (Design)



Estrutura recursiva!

Abordagem iterativa

Abordagem recursiva

Listas ligadas (paradigma funcional)

Listas ligadas (operações)

- Inserir
- Remover
- Pesquisar
- Tamanho
- Vazio

Listas ligadas (simulação)

Lista Encadeada
Abordagem Iterativa

Listas ligadas - implementação

```java
public interface LinkedList<T> {

    public void insert(T element);
    public void remove(T element);
    public T search(T element);
    public int size();
    public boolean isEmpty();
    public T[] toArray();
}
```

```java
public interface DoubleLinkedList<T>
    extends LinkedList<T> {

    public void insertFirst(T element);
    public void removeFirst();
    public void removeLast();

}
```

Listas ligadas - implementação

data	next

```
public class SingleLinkedListNode<T> {
  protected T data;
  protected SingleLinkedListNode<T> next;

  public SingleLinkedListNode() { }

  public SingleLinkedListNode(T data,
      SingleLinkedListNode<T> next) {
     this.data = data;
     this.next = next;
  }
  … gets e sets
}
```

previous	data	next

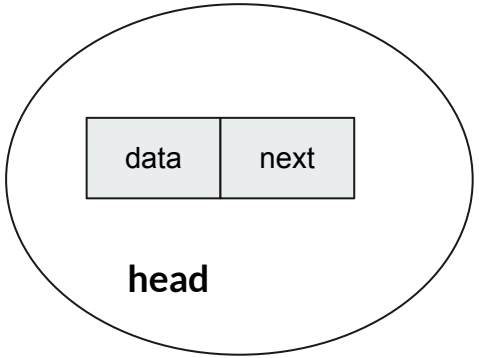
```
public class DoubleLinkedListNode<T>
      extends SingleLinkedListNode<T> {

  protected DoubleLinkedListNode<T> previous;
  public DoubleLinkedListNode() { }
  public DoubleLinkedListNode(T data,
           DoubleLinkedListNode<T> next,
           DoubleLinkedListNode<T> previous) {
    super(data, next);
    this.previous = previous;
  }

  … gets e sets
}
```

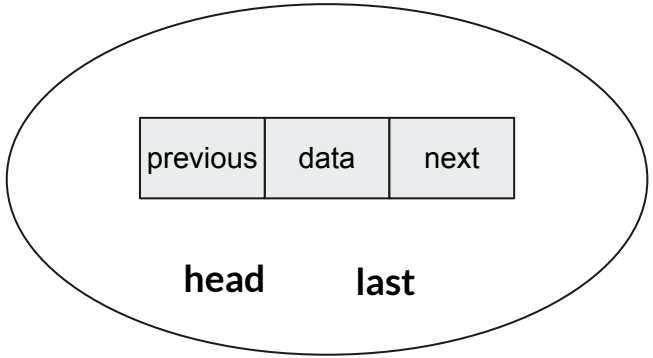
Listas ligadas - implementação

```
public class SingleLinkedListNode<T> {
  protected T data;
  protected SingleLinkedListNode<T> next;

  public SingleLinkedListNode() { }  NIL

  public SingleLinkedListNode(T data,
      SingleLinkedListNode<T> next) {
     this.data = data;
     this.next = next;
  }
  … gets e sets
}
```

```
public class DoubleLinkedListNode <T>
      extends SingleLinkedListNode<T> {

  protected DoubleLinkedListNode<T> previous;
  public DoubleLinkedListNode() { } NIL
  public DoubleLinkedListNode(T data,
           DoubleLinkedListNode<T> next,
           DoubleLinkedListNode<T> previous) {
    super(data, next);
    this.previous = previous;
  }

  … gets e sets
}
```

Listas ligadas - implementação



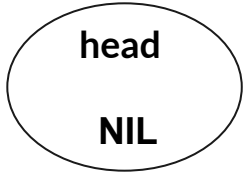


Listas ligadas (simples) - isEmpty

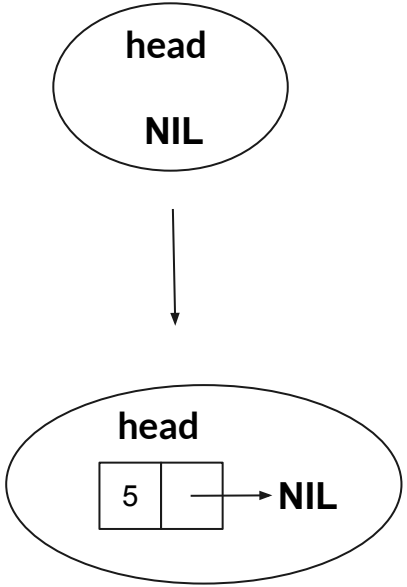
```
isEmpty( ){
  return head == NIL
}
```

Listas ligadas (simples) - insert

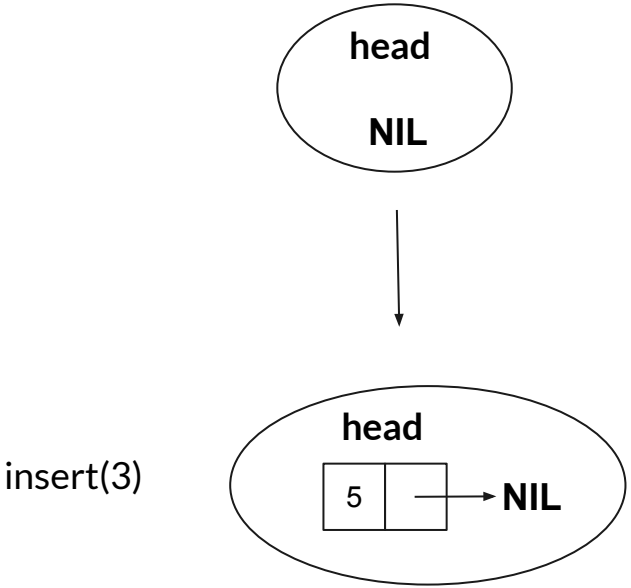
insert(5)



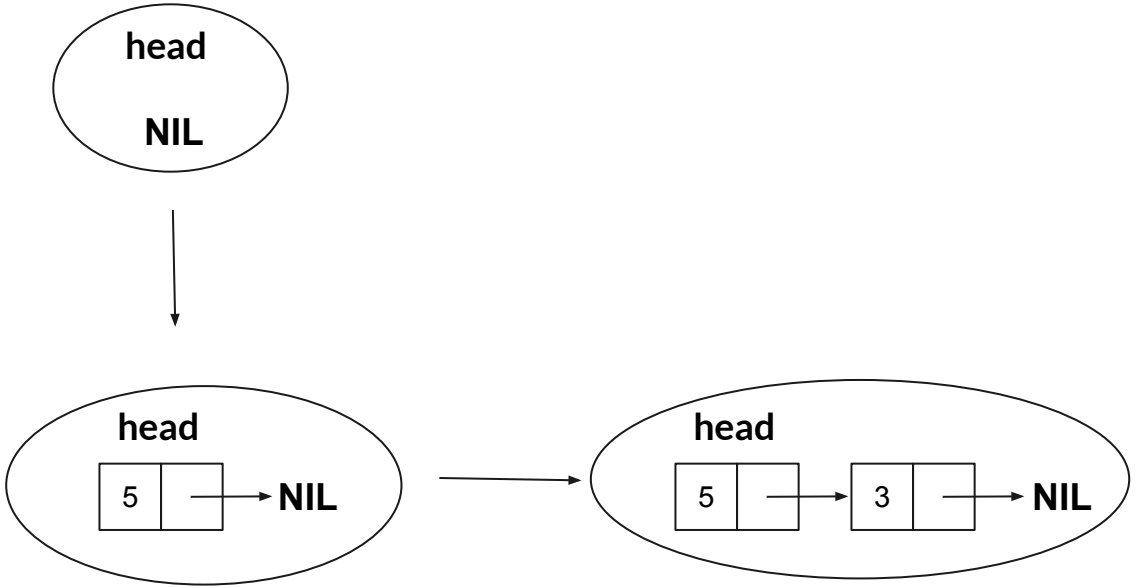
Listas ligadas (simples) - insert



Listas ligadas (simples) - insert



Listas ligadas (simples) - insert

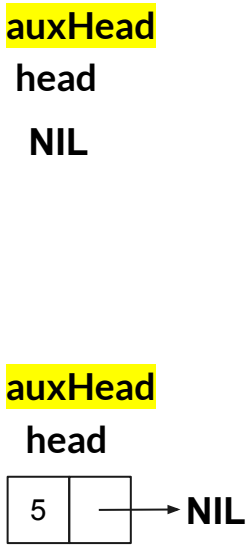


Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead.next != NIL){
       auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
 }

}
```

Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead.next != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
 }

}
```



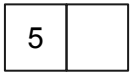
Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead.next != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
 }

}
```

auxHead

head

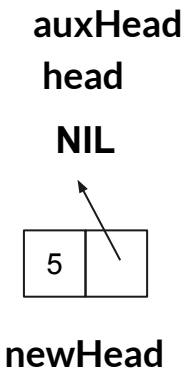
NIL



newHead

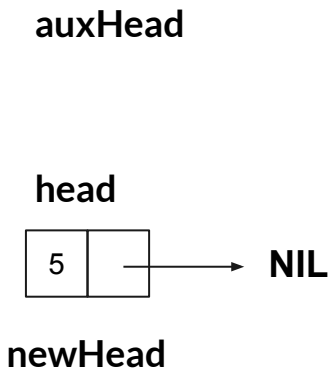
Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead.next != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
 }

}
```



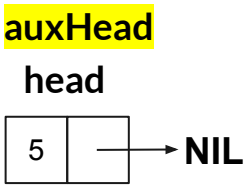
Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead.next != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
 }

}
```



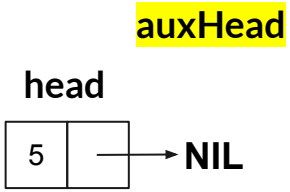
Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
  }

}
```



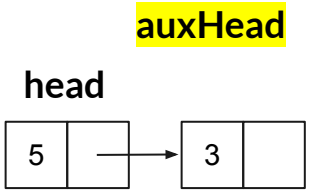
Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
  }
}
```



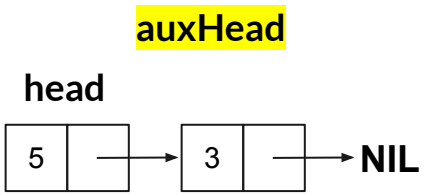
Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
  }
}
```



Listas ligadas (simples) - insert

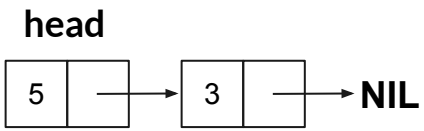
```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
  }
}
```



Listas ligadas (simples) - insert

```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
 }
}
```

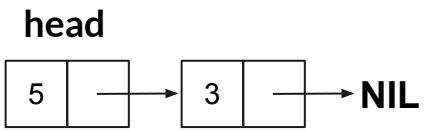
Qual a complexidade?



Listas ligadas (simples) - insert

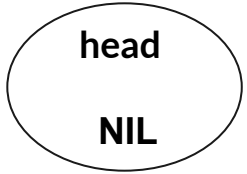
```
list-insert(item){
  auxHead = head
  if(head = NIL){
    newHead = new SingleLinkedListNode(item)
    newHead.next= head
    head = newHead
  } else{
    while(auxHead != NIL){
      auxHead = auxHead.next
    }
    auxHead.data = item
    auxHead.next = new SingleLinkedListNode()
 }
}
```

O(n)



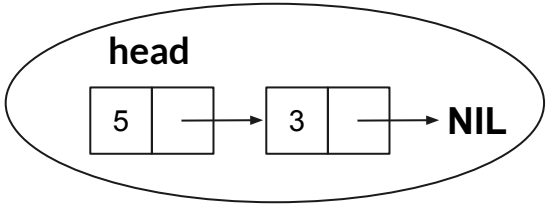
Listas ligadas (simples) - search

search(3)



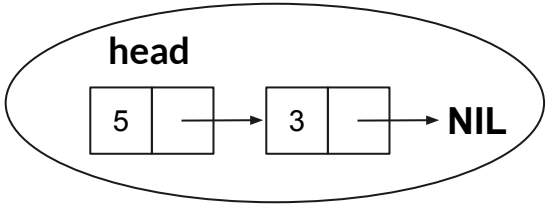
Listas ligadas (simples) - search

search(3)



Listas ligadas (simples) - search

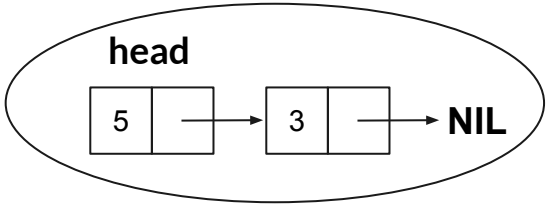
search(3)




```
list-search(item){
  auxHead = head
  while(auxHead != NIL && auxHead.data != item){
      auxHead = auxHead.next
  }
  return auxHead
}
```

Listas ligadas (simples) - search

search(3)

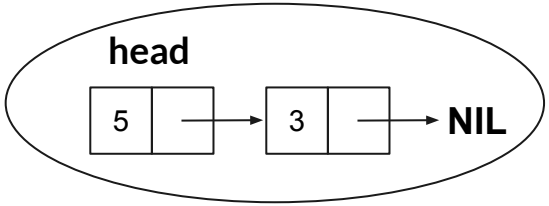


Qual a complexidade?

```
list-search(item){
  auxHead = head
  while(auxHead != NIL && auxHead.data != item){
      auxHead = auxHead.next
  }
  return auxHead
}
```

Listas ligadas (simples) - search

search(3)

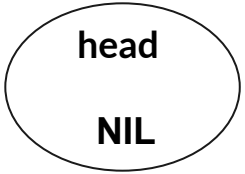


O(n)

```
list-search(item){
  auxHead = head
  while(auxHead != NIL && auxHead.data != item){
      auxHead = auxHead.next
  }
  return auxHead
}
```

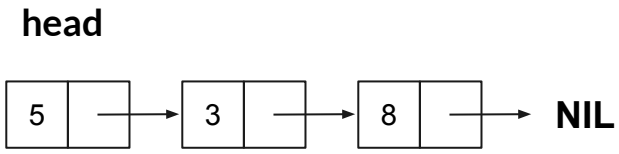
Listas ligadas (simples) - remove

remove(3)



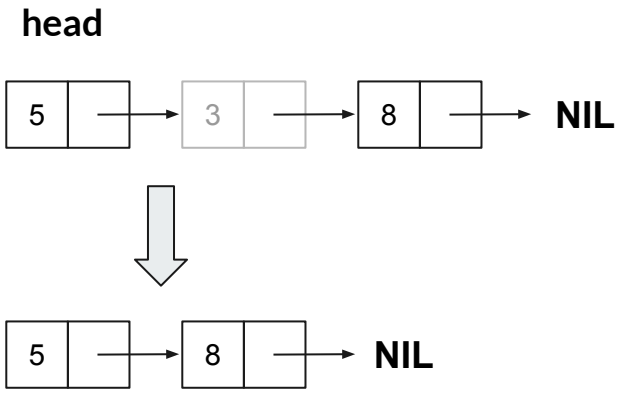
Listas ligadas (simples) - remove

remove(3)



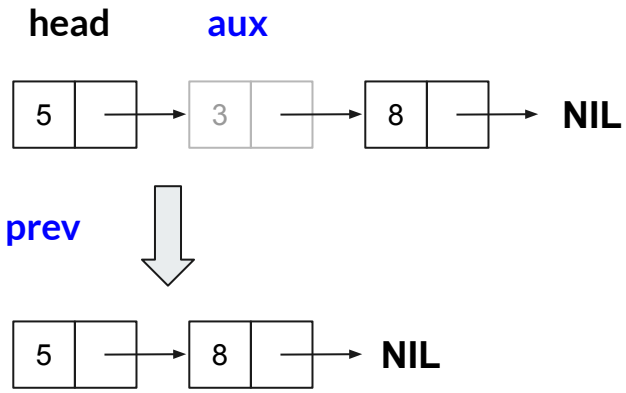
Listas ligadas (simples) - remove

remove(3)



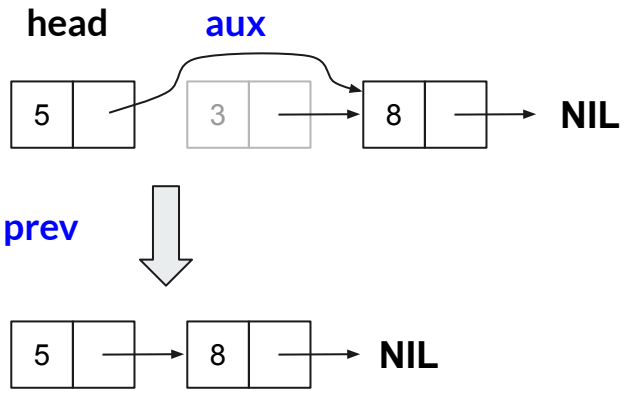
Listas ligadas (simples) - remove

remove(3)



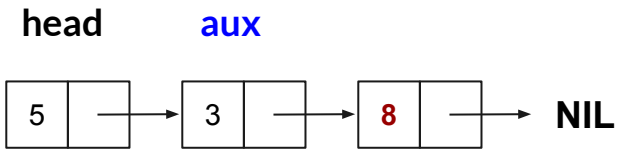
Listas ligadas (simples) - remove

remove(3)



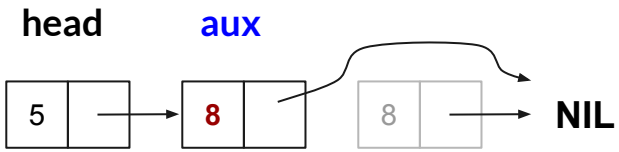
Listas ligadas (simples) - remove

remove(3)



Listas ligadas (simples) - remove

remove(3)



Listas ligadas (simples) - remove

```
list-remove(item){
  if head.data == item
     head = head.next
  else{
    aux = head
    while aux != NIL and aux.data != item {
      aux = aux.next
    }
    if aux != NIL{
     aux.data = aux.next.data
     aux.next = aux.next.next
  }
```

Listas ligadas (simples) - remove

```
list-remove(item){
  if head.data == item
     head = head.next
  else{
    aux = head
    while aux != NIL and aux.data != item {
      aux = aux.next
    }
    if aux != NIL{
     aux.data = aux.next.data
     aux.next = aux.next.next
  }
```

Qual a complexidade?

Listas ligadas (simples) - remove

```
list-remove(item){
  if head.data == item
     head = head.next
  else{
    aux = head
    while aux != NIL and aux.data != item {
      aux = aux.next
    }
    if aux != NIL{
     aux.data = aux.next.data
     aux.next = aux.next.next
  }
```

O(n)

Listas ligadas (simples) - size

```
list-size(){
  size = 0
  aux = head
  while aux != NIL {
      size = size + 1
      aux = aux.next
  }
  return size
}
```

Listas ligadas (simples) - size

```
list-size(){
  size = 0
  aux = head
  while aux != NIL {
      size = size + 1
      aux = aux.next
  }
  return size
}
```

Qual a complexidade?

Listas ligadas (simples) - size

```
list-size(){
  size = 0
  aux = head
  while aux != NIL {
      size = size + 1
      aux = aux.next
  }
  return size
}
```

O(n)

Listas duplamente ligadas

SingleLinkedListImpl

DoubleLinkedListImpl

```
public class DoubleLinkedListImpl<T>
    extends SingleLinkedListImpl<T>
     implements DoubleLinkedList<T>{

  protected DoubleLinkedListNode<T> last;

  public void insertFirst(T element)
  public void removeFirst()
  public void removeLast()

  ...
}
```

Listas duplamente ligadas

SingleLinkedListImpl

DoubleLinkedListImpl

Algo herdado precisa ser sobrescrito?

```
public class DoubleLinkedListImpl<T>
    extends SingleLinkedListImpl<T>
     implements DoubleLinkedList<T>{

  protected DoubleLinkedListNode<T> last;

  public void insertFirst(T element)
  public void removeFirst()
  public void removeLast()

  ...
}
```

Listas duplamente ligadas

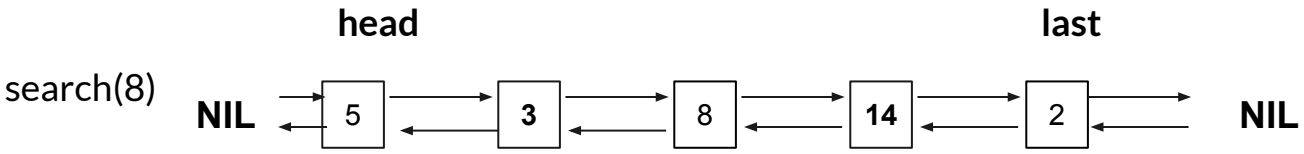
SingleLinkedListImpl

DoubleLinkedListImpl

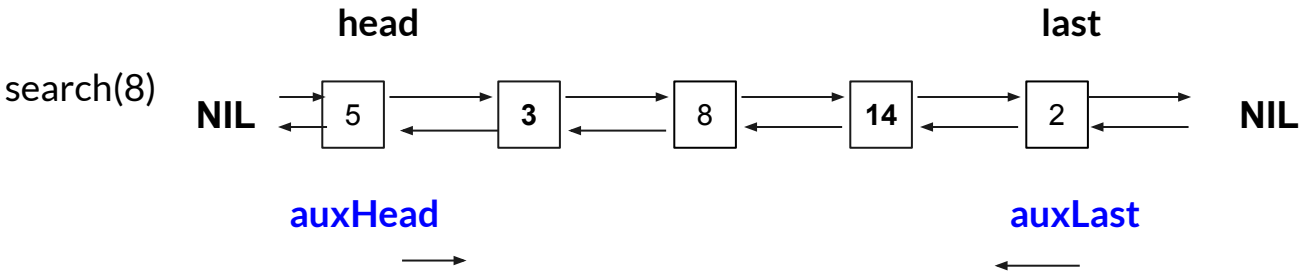
search (otimização)
insert
remove

```
public class DoubleLinkedListImpl<T>
    extends SingleLinkedListImpl<T>
     implements DoubleLinkedList<T>{

  protected DoubleLinkedListNode<T> last;

  public void insertFirst(T element)
  public void removeFirst()
  public void removeLast()

  ...
}
```

Listas duplamente ligadas - search



Listas duplamente ligadas - search



Listas duplamente ligadas - search

```
List-search(item){
  auxHead = head
  auxLast = last
  while(auxHead != auxLast and auxHead.next != auxLast
          and auxHead.data != item and auxLast.data != item){
    auxHead = auxHead.next
    auxLast = auxLast.previous
  }
  if auxHead.data == item
     return auxHead.data
  if auxLast.data == item
     return auxLast.data
}
```

Listas duplamente ligadas - search

```
List-search(item){
  auxHead = head
  auxLast = last
  while(auxHead != auxLast and auxHead.next != auxLast
          and auxHead.data != item and auxLast.data != item){
    auxHead = auxHead.next
    auxLast = auxLast.previous
  }
  if auxHead.data == item
     return auxHead.data
  if auxLast.data == item
     return auxLast.data
}
```

Qual a complexidade?

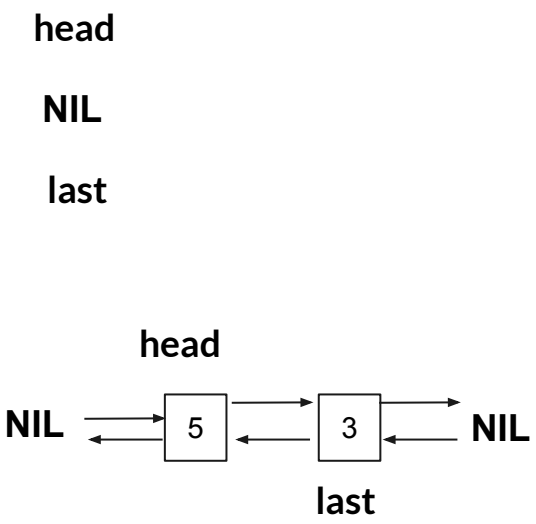
Listas duplamente ligadas - search

```
List-search(item){
  auxHead = head
  auxLast = last
  while(auxHead != auxLast and auxHead.next != auxLast
          and auxHead.data != item and auxLast.data != item){
    auxHead = auxHead.next
    auxLast = auxLast.previous
  }
  if auxHead.data == item
     return auxHead.data
  if auxLast.data == item
     return auxLast.data
}
```

O(n)

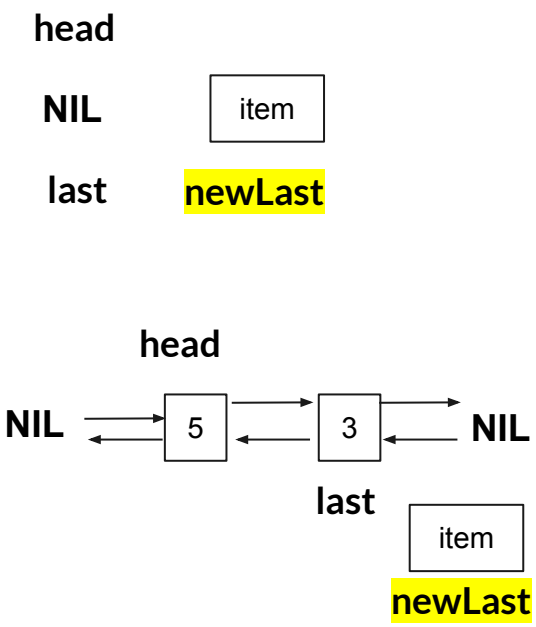
Listas duplamente ligadas - insert (final)

```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```



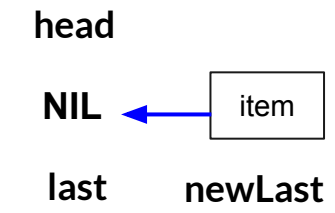
Listas duplamente ligadas - insert (final)

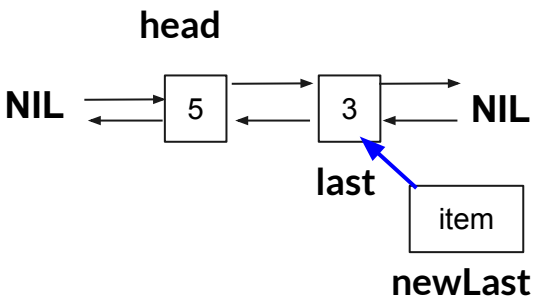
```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```



Listas duplamente ligadas - insert (final)

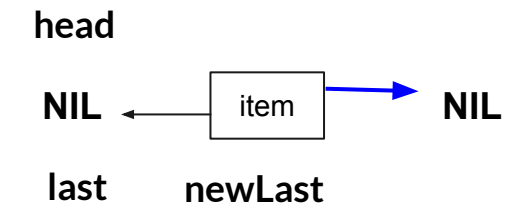
```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```

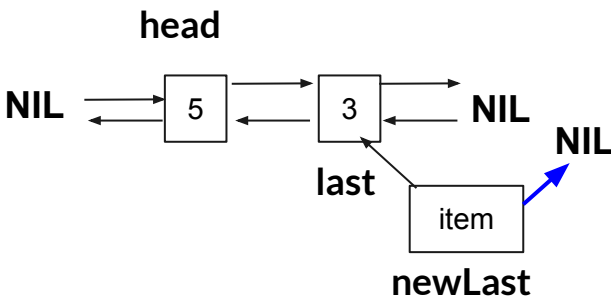




Listas duplamente ligadas - insert (final)

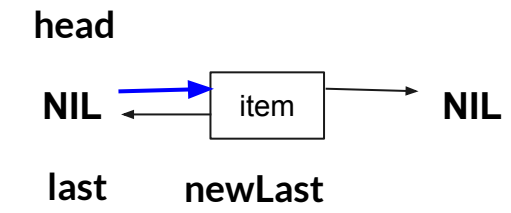
```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```

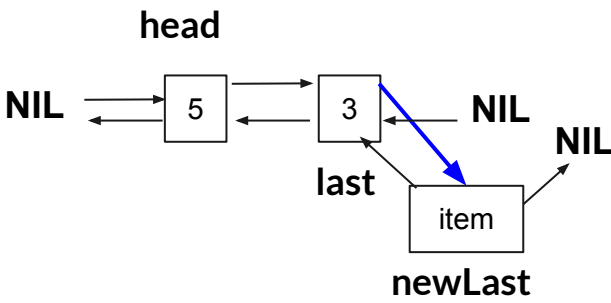




Listas duplamente ligadas - insert (final)

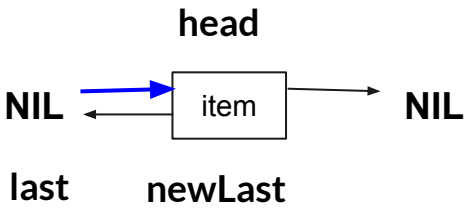
```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```

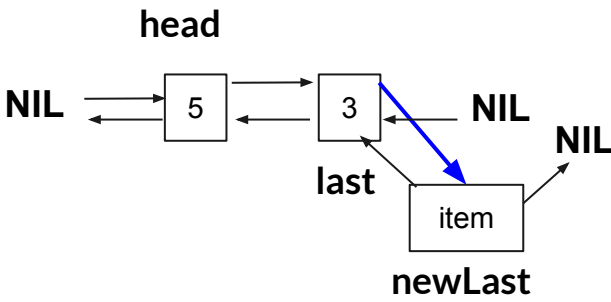




Listas duplamente ligadas - insert (final)

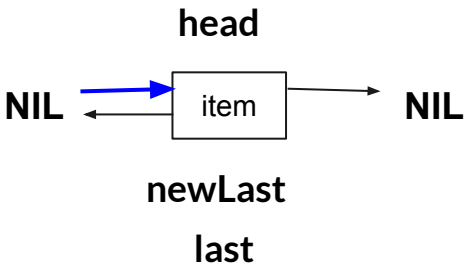
```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```

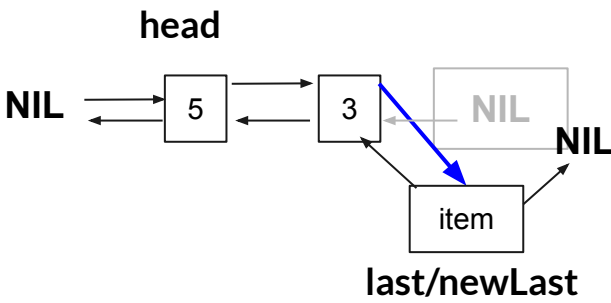




Listas duplamente ligadas - insert (final)

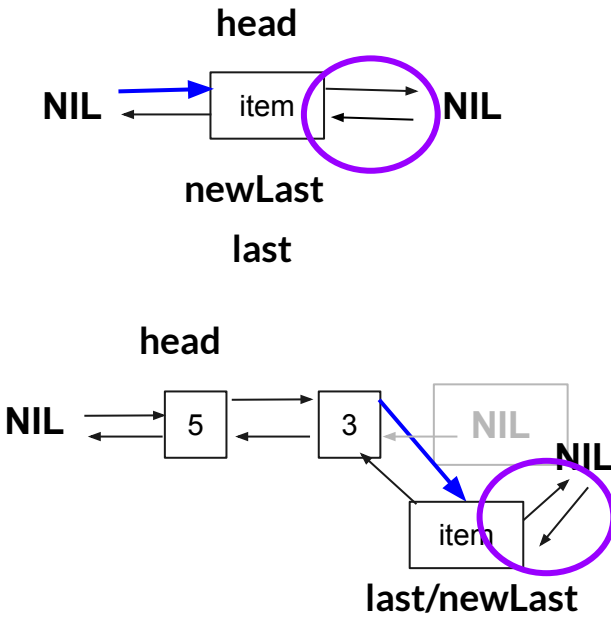
```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```





Listas duplamente ligadas - insert (final)

```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```



Listas duplamente ligadas - insert (final)

```
insertEnd(item){
  newLast = new DoubleLinkedListNode(item)
  newLast.previous = last
  newLast.next = NIL
  last.next = newLast
  if last == NIL{ //para o caso da lista vazia
    head = newLast
  }
  last = newLast
}
```

O(1)

Listas duplamente ligadas - insertFirst

```
insertFirst(item){
  newHead = new DoubleLinkedListNode(item)
  newHead.next = head
  newHead.previous = NIL
  head.previous = newHead
  if head == NIL{  //para o caso da lista vazia
    last = newHead
  }
 head = newHead
}
```

O(1)

Espelho do insertEnd, mas na outra extremidade

Listas duplamente ligadas - remove (pela chave)

```
list-remove(item){
  if head.data == item
     remove-first()
  else{
    aux = head
    while aux != NIL and aux.data != item {
      aux = aux.next
    }
    if aux != NIL{
     aux.previous.next = aux.next
     aux.next.previous = aux.previous
  }
```

Mesma lógica do remove da lista simples, exceto que a manipulação dos apontadores é simplificada

Listas duplamente ligadas - remove (pela chave)

```
list-remove(item){
  if head.data == item
     remove-first()
  else{
    aux = head
    while aux != NIL and aux.data != item {
      aux = aux.next
    }
    if aux != NIL{
     aux.previous.next = aux.next
     aux.next.previous = aux.previous
  }
```

Listas duplamente ligadas - remove (pela chave)

```
list-remove(item){
  if head.data == item
     remove-first()
  else{
    aux = head
    while aux != NIL and aux.data != item {
      aux = aux.next
    }
    if aux != NIL{
     aux.previous.next = aux.next
     aux.next.previous = aux.previous
  }
```

Posiciona aux no nó a ser removido ou NIL (final da lista)

Listas duplamente ligadas - remove (pela chave)

```
list-remove(item){
  if head.data == item
     remove-first()
  else{
    aux = head
    while aux != NIL and aux.data != item {
      aux = aux.next
    }
    if aux != NIL{
     aux.previous.next = aux.next
     aux.next.previous = aux.previous
  }
```

Se aux aponta para nó com dado então usa previous e next para ajustar a lista convenientemente

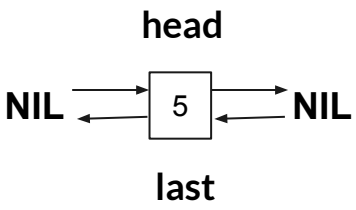
Listas duplamente ligadas - remove (pela chave)

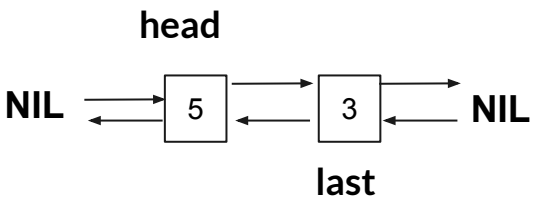
```
list-remove(item){
  if head.data == item
     remove-first()
  else{
    aux = head
    while aux != NIL and aux.data != item {
      aux = aux.next
    }
    if aux != NIL{
     aux.previous.next = aux.next
     aux.next.previous = aux.previous
  }
```

O(n)

Listas duplamente ligadas - removeFirst

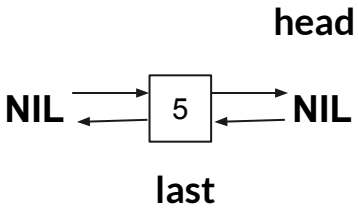
```
removeFirst(item){
  if head != NIL {
    head = head.next
    if head == NIL
      last = head
    else
      head.previous = NIL
  }
}
```

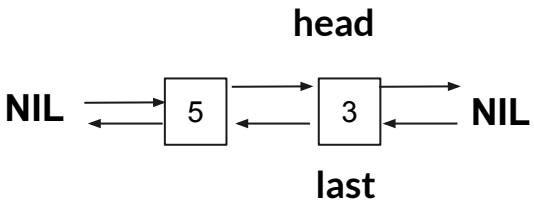




Listas duplamente ligadas - removeFirst

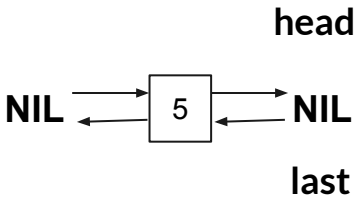
```
removeFirst(item){
  if head != NIL {
    head = head.next
    if head == NIL
      last = head
    else
      head.previous = NIL
  }
}
```

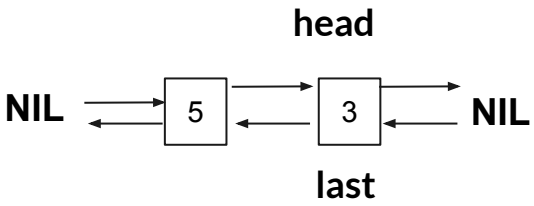




Listas duplamente ligadas - removeFirst

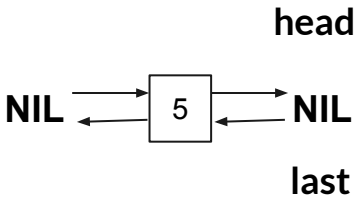
```
removeFirst(item){
  if head != NIL {
    head = head.next
    if head == NIL
      last = head
    else
      head.previous = NIL
  }
}
```

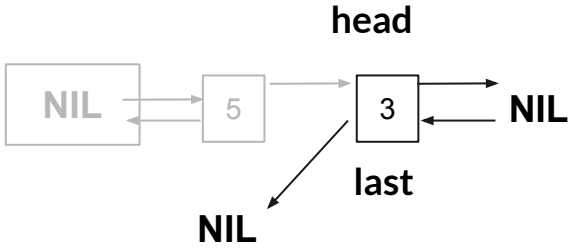




Listas duplamente ligadas - removeFirst

```
removeFirst(item){
  if head != NIL {
    head = head.next
    if head == NIL
      last = head
    else
      head.previous = NIL
  }
}
```





Listas duplamente ligadas - removeFirst

```
removeFirst(item){
  if head != NIL {
    head = head.next
    if head == NIL
      last = head
    else
      head.previous = NIL
  }
}
```

O(1)

Listas duplamente ligadas - removeEnd

```
removeEnd(item){
  if last != NIL {
    last = last.previous
    if last == NIL
      head = last
    else
      last.next = NIL
  }
}
```

Espelho do removeFirst, mas na outra extremidade

O(1)

Listas encadeadas (Java)

Class LinkedList<E>

```
java.lang.Object
    java.util.AbstractCollection<E>
        java.util.AbstractList<E>
            java.util.AbstractSequentialList<E>
                java.util.LinkedList<E>
```

Type Parameters:

`E - the type of elements held in this collection`

All Implemented Interfaces:

`Serializable, Cloneable, Iterable<E>, Collection<E>, Deque<E>, List<E>, Queue<E>`

Listas encadeadas

Aplicações e exemplos:

- Implementar uma lista ordenada (possivelmente com Comparator)
- Implementar uma pilha usando listas
- Implementar uma fila usando listas
- Implementar ordenação em uma lista ligada

Lista Encadeada
Abordagem Recursiva

Lista Encadeada

Considerar um nó como sendo a própria lista

Implementação recursiva dos métodos (maior parte)

Interface continua a mesma!

Listas ligadas - implementação (antes)

```
public interface LinkedList<T> {
  public void insert(T element);
  public void remove(T element);
  public T search(T element);
  public int size();
  public boolean isEmpty();
  public T[] toArray();
}
public interface DoubleLinkedList<T>
      extends LinkedList<T> {

  public void insertFirst(T element);
  public void removeFirst();
  public void removeLast();

}
```

```
public class SingleLinkedListImpl<T>
      implements LinkedList<T> {
  protected SingleLinkedListNode<T> head;
  ...
}
public class DoubleLinkedListImpl<T> extends
   SingleLinkedListImpl<T> implements DoubleLinkedList<T> {
  protected DoubleLinkedListNode<T> last;
  ...
}
public class SingleLinkedListNode<T> {
  protected T data;
  protected SingleLinkedListNode<T> next;
}
public class DoubleLinkedListNode<T>
      extends SingleLinkedListNode<T> {
  protected DoubleLinkedListNode<T> previous;
}
```

Listas ligadas - implementação (depois)

```
public interface LinkedList<T> {
  public void insert(T element);
  public void remove(T element);
  public T search(T element);
  public int size();
  public boolean isEmpty();
  public T[] toArray();
}
public interface DoubleLinkedList<T>
      extends LinkedList<T> {

  public void insertFirst(T element);
  public void removeFirst();
  public void removeLast();

}
```

```
public class RecursiveSingleLinkedListImpl<T>
      implements LinkedList<T> {

  protected T data;
  protected RecursiveSingleLinkedListImpl<T> next;

  public RecursiveSingleLinkedListImpl() { }
  ...
}
public class RecursiveDoubleLinkedListImpl<T> extends
    RecursiveSingleLinkedListImpl<T> implements
    DoubleLinkedList<T> {

  protected RecursiveDoubleLinkedListImpl<T> previous;

  public RecursiveDoubleLinkedListImpl() {  }
  ...
}
```

Listas ligadas - implementação (depois)

```
public interface LinkedList<T> {
  public void insert(T element);
  public void remove(T element);
  public T search(T element);
  public int size();
  public boolean isEmpty();
  public T[] to
}
public interfac
     extends Li

  public void i
  public void r
  public void removeLast();

}
```

```
public class RecursiveSingleLinkedListImpl<T>
      implements LinkedList<T> {

  protected T data;
  protected RecursiveSingleLinkedListImpl<T> next;

  public RecursiveSingleLinkedListImpl() { }

                                  ol<T> extends
                                  lements

                                  l<T> previous;

  public RecursiveDoubleLinkedListImpl() {  }
  ...
}
```

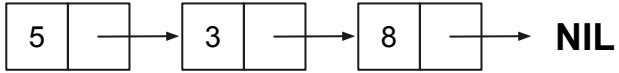
Métodos recursivos seguir o template:

- **Caso base (lista vazia/situação trivial)**
- **Caso indutivo (lista não vazia/situação não-trivial)**

Listas ligadas - implementação

NIL

caso base



caso indutivo

```
isEmpty(){
  if data == null //caso base
      return true
  else //caso indutivo
     return false
}
```

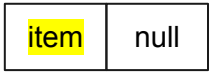
Listas ligadas - implementação

null	null

caso base

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

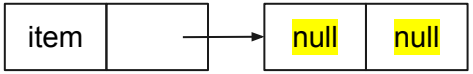
Listas ligadas - implementação



caso base

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

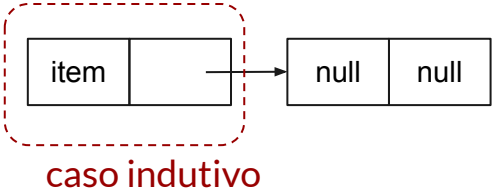
Listas ligadas - implementação



caso indutivo

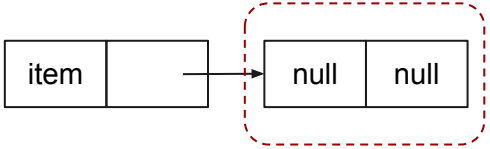
```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

Listas ligadas - implementação




```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

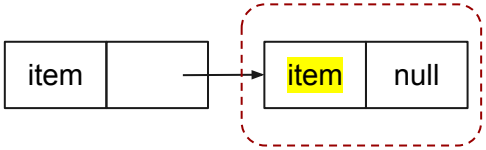
Listas ligadas - implementação



caso indutivo

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

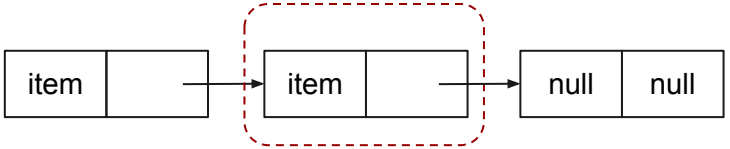
Listas ligadas - implementação



caso indutivo

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

Listas ligadas - implementação



caso indutivo

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

Listas ligadas - implementação

Qual a complexidade?

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

Listas ligadas - implementação

Qual a complexidade?

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

T(n)

O(1)

T(n - 1)

Listas ligadas - implementação

Qual a complexidade?

$$T(n) = T(n - 1) + O(1) \Rightarrow \theta(n)$$

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = new RecursiveSingleLinkedListImpl<T>( )
   else //caso indutivo
      next.list-insert(item)
}
```

T(n)

O(1)

T(n - 1)

Listas ligadas - implementação

```
list-search(item){
   if(isEmpty) //caso base

   else




}
```

Listas ligadas - implementação

```
list-search(item){
  if(isEmpty) //caso base
    return null
  else
    if data==item //caso base
      return data
    else //caso indutivo
      return next.list-search(item)
}
```

Listas ligadas - implementação

```
list-search(item){
  if(isEmpty) //caso base
    return null
  else
    if data==item //caso base
      return data
    else //caso indutivo
      return next.list-search(item)
}
```

$T(n) = T(n - 1) + O(1) \Rightarrow \theta(n)$

Listas ligadas - implementação

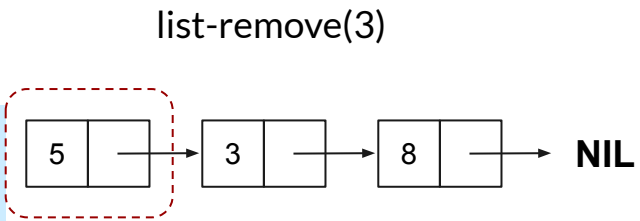
```
list-remove(item){
   if(isEmpty) //caso base
      //nao faz nada porque nao contem o elemento
   else //caso indutivo




}
```

Listas ligadas - implementação

```
list-remove(item){
   if(isEmpty) //caso base
      //nao faz nada porque nao contem o elemento
   else
         if data==item //caso base
         data = next.data
         next = next.next
      else //caso indutivo
         next.list-remove(item)
}
```

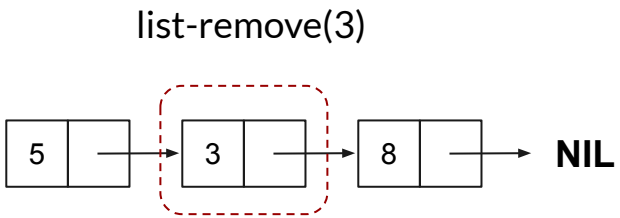
Listas ligadas - implementação

```
list-remove(item){
   if(isEmpty) //caso base
      //nao faz nada porque nao contem o elemento
   else
         if data==item //caso base
         data = next.data
         next = next.next
      else //caso indutivo
         next.list-remove(item)
}
```



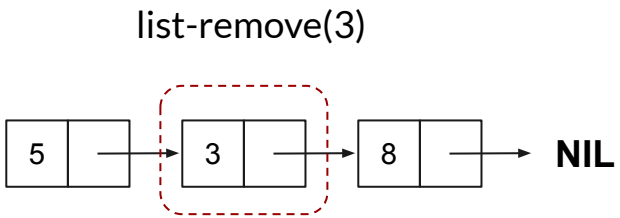
Listas ligadas - implementação

```
list-remove(item){
   if(isEmpty) //caso base
      //nao faz nada porque nao contem o elemento
   else
         if data==item //caso base
         data = next.data
         next = next.next
      else //caso indutivo
         next.list-remove(item)
}
```



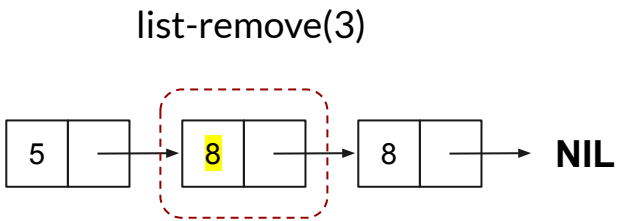
Listas ligadas - implementação

```
list-remove(item){
   if(isEmpty) //caso base
      //nao faz nada porque nao contem o elemento
   else
         if data==item //caso base
         data = next.data
         next = next.next
      else //caso indutivo
         next.list-remove(item)
}
```



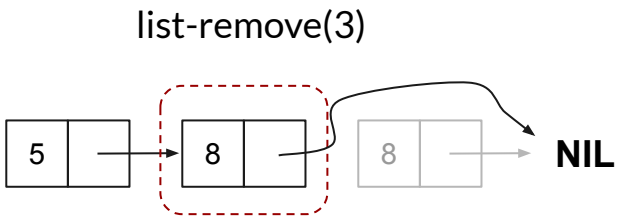
Listas ligadas - implementação

```
list-remove(item){
   if(isEmpty) //caso base
      //nao faz nada porque nao contem o elemento
   else
         if data==item //caso base
         data = next.data
         next = next.next
      else //caso indutivo
         next.list-remove(item)
}
```



Listas ligadas - implementação

```
list-remove(item){
   if(isEmpty) //caso base
      //nao faz nada porque nao contem o elemento
   else
         if data==item //caso base
         data = next.data
         next = next.next
      else //caso indutivo
         next.list-remove(item)
}
```



Listas ligadas - implementação

```
list-remove(item){
   if(isEmpty) //caso base
      //nao faz nada porque nao contem o elemento
   else
         if data==item //caso base
         data = next.data
         next = next.next
      else //caso indutivo
         next.list-remove(item)
}
```

$T(n) = T(n - 1) + O(1) \Rightarrow \theta(n)$

Listas ligadas - implementação

```
list-size(){
   if(isEmpty) //caso base

   else //caso indutivo

}
```

Listas ligadas - implementação

```
list-size(){
   if(isEmpty) //caso base
     return 0
   else //caso indutivo
     return 1 + next.list-size()
}
```

$T(n) = T(n - 1) + O(1) \Rightarrow \theta(n)$

Listas ligadas - implementação

```
T[] list-toArray(){
   T[] result = new java.util.LinkedList()
   toArray(result,this)
   return (T[]) result.toArray()
}

toArray (LinkedList list, RecursiveSingleLinkedListImpl node){
   if (!node.isEmpty){
      list.add(node.data)
      toArray(list,node.next)
   }
}
```

Listas ligadas - implementação

Implemente um método que encontra o maior elemento de uma lista ligada

Listas ligadas - implementação

Implemente um método que encontra o maior elemento de uma lista ligada

```
maior(){
  if(isEmpty) //caso base
    return 0
  else //caso indutivo
    maior = this.data
    maiorNext = next.maior()
    if(maiorNext > maior)
        maior = maiorNext
    return maior
}
```

Listas duplamente ligadas

RecursiveSingleLinkedListImpl

RecursiveDoubleLinkedListImpl

insert
remove

```
public class RecursiveSingleLinkedListImpl<T>
      implements LinkedList<T> {

  protected T data;
  protected RecursiveSingleLinkedListImpl<T> next;

  public RecursiveSingleLinkedListImpl() { }
  ...
}
public class RecursiveDoubleLinkedListImpl<T> extends
    RecursiveSingleLinkedListImpl<T> implements
    DoubleLinkedList<T> {

  protected RecursiveDoubleLinkedListImpl<T> previous;

  public RecursiveDoubleLinkedListImpl() {  }
  ...
}
```

Listas duplamente ligadas - insert (herdado)

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = NIL
   else //caso indutivo
      next.list-insert(item)
}
```

Listas duplamente ligadas - insert (sobrescrito)

```
list-insert(item){
   if(isEmpty) //caso base
      data = item
      next = NIL
     if (previous == null)
         previous = NIL
   else //caso indutivo
      next.list-insert(item)
}
```

Listas duplamente ligadas - remove (herdado)

```
list-remove(item){
  if(isEmpty) //caso base
    //nao faz nada porque nao contem o elemento
  else
    if data==item //caso base
      data = next.data
      next = next.next
    else //caso indutivo
      next.list-remove(item)
}
```

Listas duplamente ligadas - remove (sobrescrito)

```
list-remove(item){
  if(isEmpty) //caso base
    //nao faz nada porque nao contem o elemento
  else
    if data==item //caso base
      if(previous.isEmpty && next.isEmpty){
        data = next = previous = null
      else
        data = next.data
        next = next.next
        if(next != null)
          next.previous = this
    else //caso indutivo
      next.list-remove(item)
}
```

Listas duplamente ligadas

```
public void insertFirst(T element)
public void removeFirst();
public void removeLast()
```

Quais precisam ser recursivos?

Listas duplamente ligadas

```
public void insertFirst(T element)
public void removeFirst();
public void removeLast()
```

Quais precisam ser recursivos?

Implementações deixadas como exercício